txn-repr-poc - a maintainer's guide

Setting up, running, and not breaking the transaction-representation prototype. It replicates Raman, Ganesh and Veloso, arXiv:2410.07851 (NeurIPS 2024), on synthetic ISO 20022 payments.

Start here

You can run the whole project on a laptop, with no GPU and no language model downloaded. For something built around a 1.3-billion-parameter model that sounds wrong, and it's the first thing worth knowing, because it's what makes the edit loop fast. Two commands:

```
python run_gpu.py --smoke --limit 2000
pytest -q
```

Between them they drive every layer - generator, encoder, decoder, scorer - on CPU in well under a minute. The reason it works is one seam: the decoder talks to the language model through a small interface (`LLMInterface`), and in smoke mode a 32-dimension MockLLM stands in for Phi-1.5. Nothing downloads. The 95 tests use the same stand-in, so a fresh checkout is green before you've installed `transformers` or touched a GPU. That's the property you want while changing code: a one-minute answer to whether you broke something.

What you need installed

Day to day, you need Python and seven packages. The GPU and the real model are for the headline run only; you can maintain this code for weeks without either.

What	Version	Note
Python	3.10 - 3.12	developed on 3.10.11
git, pip, venv	recent	stdlib venv is fine
requirements.txt	numpy, pandas, pyarrow, pyyaml, torch, scikit-learn, catboost	the torch CPU wheel is enough for dev
pytest	any	not pinned in requirements - install it yourself

Three more are optional and easy to over-install. `transformers` pulls in the real Phi-1.5 and matters only for the GPU run or for scoring a real checkpoint - the CPU loop never imports it. A CUDA GPU (the headline run used one H200, fp32) turns the real run from overnight into minutes. `huggingface_hub` plus a token only matters if you're pulling the published weights, which MODEL.md covers. Install them when you reach for them.

Setup, start to finish

```
git clone https://github.com/Bratz/txn-repr-poc.git
cd txn-repr-poc
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install pytest
# only for the real model run / scoring real checkpoints:
# pip install transformers huggingface_hub
```

Then prove the environment before you trust it:

```
python data/synth_pacs008.py --parents 4000 --transactions 200000 \
  --out data/pacs008_synth.parquet --schema-out data/column_schema.json
python run_gpu.py --smoke --limit 2000
pytest -q
```

Green tests mean you're set. Skip the data step and the tests still pass - they fall back to the committed `pacs008_sample_500.csv` and `column_schema.example.json`. That's deliberate: a clone should be testable before it has generated a single row.

How the pieces fit

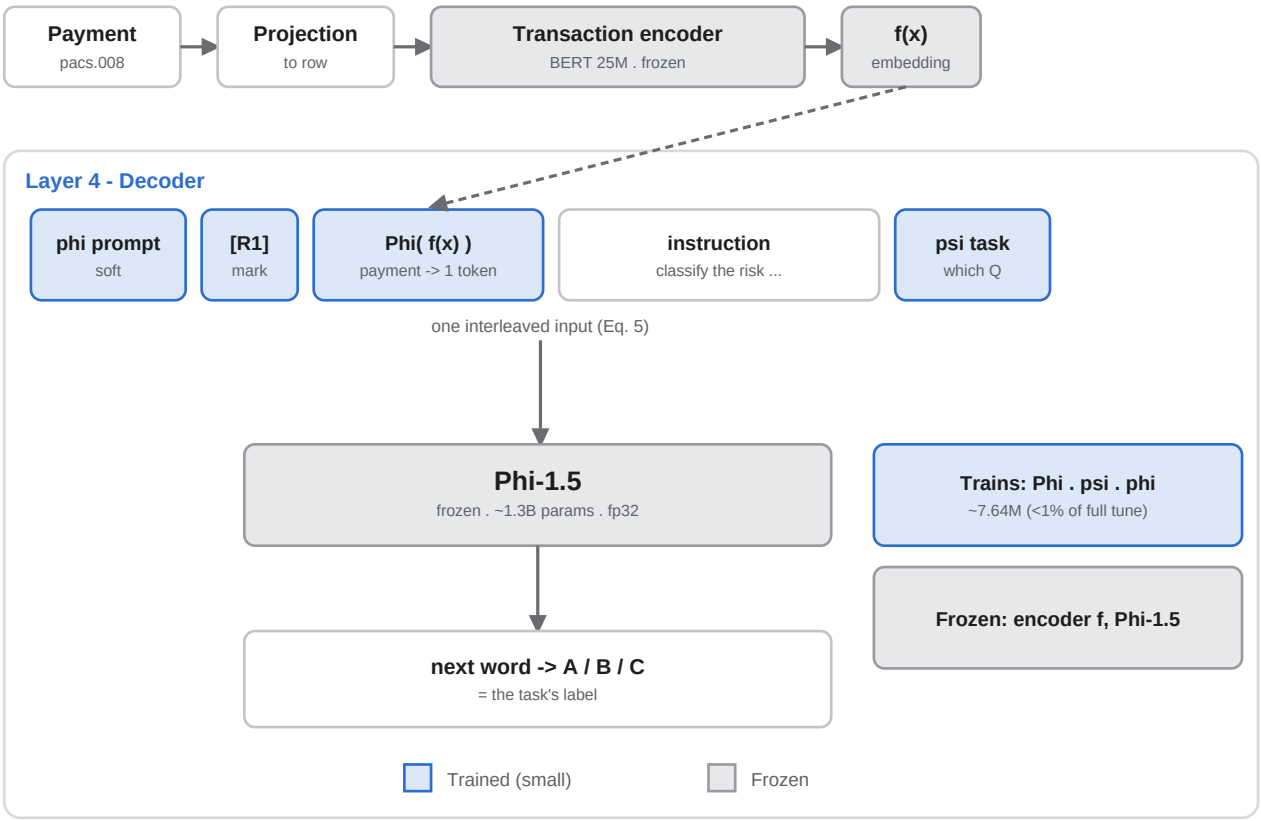
Every module names the paper section it implements in a comment at the top. Read them top to bottom - that's the order the data flows.

Path	Role
`data/synth_pacs008.py`	Layer 1: the Algorithm-1 generator, the pacs.008 projection, the four task labels, and the schema 'tasks' manifest
`encoders/partitioning_embedder.py`	Sec 3.1 - frequency-split embedding for high-card account IDs
`encoders/quantizer.py`	Sec 3.3 - currency-conditioned amount quantizer
`encoders/party_encoder.py`	Sec 3.2 - offline party encoder + the persistent party store
`encoders/column_assembler.py`	Eq. 4 - routes each column to its encoder, builds the token sequence
`encoder/tabular_encoder.py`	Sec 3.4 - BERT (25M) with the reconstruction + batch-hard-triplet loss
`decoder/multimodal_decoder.py`	Sec 4 - frozen f + frozen LLM + trainable {Phi, psi, phi}; multi-record Eq. 5
`eval/metrics.py, baselines.py`	Layer 5 - per-task metrics and the CatBoost baseline
`run_gpu.py`	the orchestrator (see below)
`predict.py`	online scoring - save/load a checkpoint, score by task
`configs/default.yaml`	pinned hyperparameters + the falsifiable claims and thresholds
`data/column_schema.json`	the contract everything reads - buckets + task manifest

Two files carry more than their names admit. `run_gpu.py` builds and freezes the encoder (the C1 experiment), then instruction-tunes the adapters across all four tasks and scores them against CatBoost (C2), and writes results.json. `column_schema.json` is the contract: buckets and the task list live there, and nothing downstream may hard-code a column list. Break that one rule and a schema change silently stops reaching the code that depends on it.

The design in one picture

Here's how a payment becomes an answer at run time. The encoder turns a payment into one embedding; the adapter Phi makes that embedding a single token a frozen Phi-1.5 reads alongside the instruction and the task signal. Only the blue pieces train.



Running the real thing

The full run swaps the MockLLM for a frozen Phi-1.5 and writes results.json. `--save-dir` persists a scorer; `--full-tune` adds the unfrozen-LLM comparator for C2.

```
python run_gpu.py --save-dir ckpt
python run_gpu.py --full-tune --save-dir ckpt
```

Scoring a saved model is `predict.py`. Single-record tasks score per row; recurrence scores per debtor-creditor group, because recurrence is a pattern across several payments rather than a property of one.

```
python predict.py --model-dir ckpt --input new_rows.parquet \
  --out scored.csv --task risk     # or geography | expense | recurrence
```

One trap, and it has caught people. The predict CLI rebuilds a real language model from the name stored in the checkpoint. Point it at a smoke checkpoint, whose model is 'mock', and it tries to download a model called mock and dies. Smoke checkpoints can only be scored through the Python API with a MockLLM passed in - `load_model(dir, llm=MockLLM(...))`, which is what the tests do. Worth saying plainly: whether the four-task numbers hold at production scale, nobody knows yet - the published checkpoint is the risk-only run, and the multi-task GPU pass hasn't been done.

The thing most likely to break - the freeze invariant

If you break one thing in this repo by accident, it'll be the freeze. At Layer 4 the encoder f and the language model are frozen; only three small modules train - the adapter Phi, the task embedding psi, and the prompt parameters phi. The headline result depends on it: unfreeze either the encoder or the LLM and you haven't tuned the model, you've run a different experiment, which means the C1 and C2 numbers in RESULTS.md

stop describing the thing you're running and start describing something that no longer exists. There's an `assert_frozen()` guard, but it only fires if you call it. Treat the frozen state as load-bearing.

Five more rules travel with the code, for the same reason: the value here is fidelity to the paper, not better metrics. Don't retune the pinned hyperparameters (B=4, alpha_v=-3, alpha_d=2.25; a 25M encoder, 3 epochs). Read buckets and tasks from column_schema.json, never from a list in the code. Keep to the three sanctioned departures - the pacs.008 schema, currency-conditioned quantization, imbalance-aware metrics - and raise a fourth out loud instead of slipping it in. Leave the walked-back extensions walked back: the completeness vector, the structuring/layering chain task, the held-out-typology split - and don't confuse that chain task with recurrence, which is a paper task and belongs here. When unsure, do what the paper did and leave a '# PAPER: section x.y' comment so the next person can check you.

Footguns

A few things will cost you an afternoon if nobody warns you.

results*.json are run artifacts, not records. They're gitignored and regenerated by the runners; RESULTS.md and docs/V2_DIRECTION.md are the quotable record. If a results file disagrees with the docs, re-run before quoting either.

Phi loads in fp32 on purpose. The adapter and prompt vectors are fp32, and an fp16 Phi throws at its first LayerNorm when they meet. If you 'save memory' by switching it to fp16, that's the crash you'll get.

Generated files aren't in the clone. column_schema.json and the parquet are gitignored - regenerate them. The committed example files are what the tests fall back to. And on Windows, git's LF-to-CRLF warnings are just noise.

Changing things

Adding a fifth task is the test of whether you've understood the shape. You write a label rule and a manifest entry in synth_pacs008.py; `build_task_specs` in run_gpu.py reads the manifest and wires the instruction tokens; predict.py scores it by name. A multi-record task sets `records='multi'` and names its group column. You shouldn't have to touch the encoder, the decoder internals, or the LLM. If you do, stop - that's usually the freeze invariant about to be broken.

The whole job comes down to three habits: keep the one-minute CPU loop green, keep f and Phi frozen, and keep every claim traceable to a paper section or a sanctioned departure. For anything deeper, architecture.md is the source of truth on what each component is, and the module you need usually has the comment that answers your question at the top of the file.